

Exploring FPGA designs for MX and beyond

Ebby Samson

Imperial College London

London, UK

es1219@ic.ac.uk

Naveen Mellempudi

AMD

Austin, USA

naveen.mellempudi@amd.com

Wayne Luk

Imperial College London

London, UK

w.luk@imperial.ac.uk

George A. Constantinides

Imperial College London

London, UK

g.constantinides@imperial.ac.uk

Abstract—A number of companies recently worked together to release the new Open Compute Project MX standard for low-precision computation, aimed at efficient neural network implementation. In this paper, we describe and evaluate the first open-source FPGA implementation of the arithmetic defined in the standard. Our designs fully support all the standard’s concrete formats for conversion into and out of MX formats and for the standard-defined arithmetic operations, as well as arbitrary fixed-point and floating-point formats. Certain elements of the standard are left as implementation-defined, and we present the first concrete FPGA-inspired choices for these elements, which we outline in the paper. Our library of optimized hardware components is available open source, and can be used to build larger systems. For this purpose, we also describe and release an open-source Pytorch library for quantization into the new standard, integrated with the Brevitas library so that the community can develop novel neural network designs quantized with MX formats in mind. We demonstrate the usability and efficacy of our libraries via the implementation of example neural networks such as ResNet-18 on the ImageNet ILSVRC12 dataset. Our testing shows that MX is very effective for formats such as INT5 or FP6 which are not natively supported on GPUs. This gives FPGAs an advantage as they have the flexibility to implement a custom datapath and take advantage of the smaller area footprints offered by these formats.

Index Terms—MX, FPGA, Brevitas, quantization, scale

I. INTRODUCTION

Developments in digital hardware over the past few decades have greatly increased the compute performance available for machine learning training and inference. This has allowed larger neural networks with more parameters to be trained and deployed, yielding ever more powerful models with better accuracy. However, the demand for more computational power is still increasing with demand for models with more parameters [1]. Aside from this, there is demand for the ability to deploy powerful models with large numbers of parameters in low-power environments such as edge devices. For these reasons, lots of research in digital hardware design had the goal of creating hardware which allows deploying these models with smaller footprints while preserving inference accuracy.

Traditionally, machine learning inference and training are performed using the IEEE FP32 format, with all values such as parameters, activations, gradients and weight updates represented in FP32. However, there has been a growing demand for compact data representations with high-throughput compute. The MX standard [2] aims to help create more compact models while preserving the accuracy of their full-precision counterparts. The standard introduces quantization with a new

scale sharing regime, as compared to traditional per-tensor or per-channel quantization. Our contributions are the following:

- We present the first open-source FPGA-oriented implementation of the new MX standard.
- We explore parameter choices beyond the concrete implementations defined in the standard and evaluate their impact on inference accuracy and FPGA area, allowing us to uncover some promising design points.
- We provide open-source software infrastructure to facilitate exploration with MX and similar schemes in Pytorch.

II. BACKGROUND

A. Quantization

One common approach to reducing area and bandwidth of neural networks is by exploring number representations where fewer bits are used per parameter than in IEEE 754 [3] floating-point formats. A downside of these narrow formats is that they typically have lower precision or dynamic range in comparison. The reduction in dynamic range is extreme in floating-point formats with narrow exponents [4] and fixed-point formats [5]. These formats are usually coupled with a shared scale factor and zero point [6] which transform values to make better use of the available range. This transformation is shown in Equation 1 where tensors S and Z represent the scale and zero point respectively and $\odot$ represents element-wise multiplication. Equation 2 shows restrictions imposed by the MX standard, where $\mathbb{Z}_b$ denotes the set of values representable by a b -bit integer and $\mathbb{M}_{e,m}$ denotes the set of values representable by a floating-point format with an e -bit exponent and m -bit mantissa. Quantized tensor X_q can be integer or floating-point. The scale and zero point scale and shift values such that most of them lie in the range representable by the target format, and that most of them have distinct values in the target format. They are usually shared with tensor-wise or channel-wise granularity due to the computational cost savings from factoring them out of dot products. Table I shows the restrictions imposed by scale sharing regimes on a 2D convolution (Equation 3). Per-tensor scales restrict all elements of the scale to the same value, while per-channel scales allow scales to vary along the C dimension. In the case of per-vector [7] and MX [8] scaling, the scale tensor is reshaped such that the C dimension is replaced with two dimensions $\lceil \frac{C}{k} \rceil$ and k . In this configuration, only values along the k dimension share a common scale. The need for reshaping will be explained in Section IV-A.